Task 1

Evaluation of Chatbot Interfaces and Detection Approaches

Introduction

Detecting AI chatbot interfaces via computer vision is required as tools like ChatGPT, Gemini, and Claude become embedded in desktop workflows. This analysis assesses their visual design and the implications of interface evolution (UI drift) on detection. Subsequently, it evaluates potential computer vision methodologies to establish a justified approach.

Visual Review of Target Chatbot Systems

ChatGPT

The ChatGPT web interface follows a structured two panel layout. A left hand sidebar is used for chat history and navigation, while the main conversation area occupies the central portion of the screen. Messages are displayed vertically, and a persistent text input field is positioned at the bottom of the interface (OpenAI, 2024). ChatGPT supports both light and dark modes, which primarily affect background colour and text contrast while preserving the overall layout.

Visually, ChatGPT is characterised by minimal branding, rounded message containers, and a neutral colour palette. However, the interface has undergone incremental updates over time, including changes to sidebar behaviour, spacing, and icon placement (OpenAI, 2024). These changes introduce UI drift, meaning that models trained on earlier interface versions may experience reduced accuracy if they rely on fixed visual features rather than more general layout patterns.

Gemini

Gemini presents a chat centric interface with a central conversation panel and a prominent prompt input area. Navigation elements are integrated into side or top level panels, following Google's Material Design principles (Google, 2024). Gemini supports both light and dark themes and typically adapts automatically to the system's appearance settings.

Compared to ChatGPT, Gemini exhibits stronger visual branding, including Google typography and colour accents. The Gemini interface has undergone visible redesigns, such as modifications to the prompt bar and the introduction of additional navigation features (9to5Google, 2025). These frequent changes increase visual variability and highlight the risk of UI drift for detection models trained on static screenshots.

Claude

Claude adopts a conversational interface similar to the other systems but introduces additional complexity through features such as Artifacts. Artifacts display longer outputs, including code or documents, in a separate panel adjacent to the main chat, resulting in a multi panel layout (Anthropic, 2024).

Claude supports light and dark appearance modes, which alter contrast and background colour while maintaining structural consistency. The introduction of additional panels and evolving interface elements increases layout diversity, which can pose challenges for visual detection systems that are not designed to handle variable screen compositions (Anthropic, 2024).

Computer Vision Approaches for Chatbot Detection

Image Classification

Image classification assigns a single label to an entire image. While this approach is computationally efficient, it assumes that the chatbot interface dominates the image. In realistic desktop screenshots where multiple applications may be visible simultaneously, image classification lacks spatial awareness and cannot localise specific chatbot windows. As a result, this approach is not well suited to the requirements of this task (He et al., 2016).

Object Detection

Object detection techniques predict both the class and location of objects within an image. This makes them well suited to chatbot detection in complex desktop environments where chat windows may be resized, repositioned, or partially occluded. Modern one stage detectors such as YOLO provide an effective balance between accuracy and inference speed, making them suitable for both screenshot based and video based detection (Redmon et al., 2016; Ultralytics, 2023).

Template Matching and UI Element Recognition

Template matching and feature based methods attempt to identify fixed UI components such as logos or icons. While these techniques can be effective in controlled environments, they are highly sensitive to changes in scale, colour, and interface layout. Given the frequent redesigns and theme variations of chatbot interfaces, template based approaches are not robust enough for long term deployment (Lowe, 2004).

Data Augmentation Techniques

Data augmentation improves model generalisation by increasing the diversity of training data. Geometric augmentations such as scaling, translation, and cropping simulate window resizing and partial occlusion, while photometric augmentations model appearance changes caused by light and dark themes. Augmentation is particularly important for mitigating the effects of UI drift and improving robustness to real world usage scenarios (Cubuk et al., 2020).

Method Selection and Justification

Considering the dynamic nature of chatbot interfaces and the complexity of real desktop environments, object detection combined with targeted data augmentation is the most appropriate approach for this assignment. Unlike image classification, object detection enables precise localisation of chatbot windows, and unlike template matching, it remains effective under theme changes, partial occlusion, and interface redesigns.

By learning spatial and structural features rather than relying on fixed UI templates, object detection models are better equipped to handle UI drift. Data augmentation further strengthens this approach by exposing the model to realistic variations during training, improving long term robustness (Ultralytics, 2023).

Conclusion

This section has reviewed the visual characteristics of ChatGPT, Gemini, and Claude, highlighting both shared design conventions and platform specific features. While all three systems adopt a chat based interaction model, differences in layout complexity, branding, and update frequency introduce challenges for visual detection.

Through comparison of multiple computer vision techniques, object detection has been identified as the most suitable approach for detecting chatbot interfaces in screenshots and video frames. When combined with appropriate data augmentation strategies, this approach provides resilience to interface variation and UI drift, forming a strong foundation for the detection system developed in subsequent tasks.

References

Anthropic (2024) Introducing Claude 3.5 Sonnet and Artifacts. Available at: https://www.anthropic.com/news (Accessed: 2025).

Cubuk, E.D., Zoph, B., Shlens, J. and Le, Q.V. (2020) 'RandAugment: Practical automated data augmentation with a reduced search space', Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, pp. 702–703.

Google (2024) Use dark theme in Gemini apps. Available at: https://support.google.com (Accessed: 2025).

He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778.

Lowe, D.G. (2004) 'Distinctive image features from scale-invariant keypoints', International Journal of Computer Vision, 60(2), pp. 91–110.

OpenAI (2024) ChatGPT appearance and interface settings. Available at: https://help.openai.com (Accessed: 2025).

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) 'You Only Look Once: Unified, real-time object detection', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 779–788.

Ultralytics (2023) Ultralytics YOLOv8 documentation. Available at: https://docs.ultralytics.com (Accessed: 2025).

9to5Google (2025) 'Gemini gets web redesign with new dark theme and navigation features'. Available at: https://9to5google.com (Accessed: 2025).

Task 2

Introduction

Object detection systems are highly sensitive to geometric variations in object scale, position, and spatial context. In real-world desktop environments, chatbot interfaces such as ChatGPT, Google Gemini, and Anthropic Claude are rarely presented in a fixed, full-screen layout. Instead, they are frequently resized, repositioned, partially obscured, or embedded among other application windows. To account for these conditions, geometric and photometric data augmentation techniques were applied to the dataset in order to improve robustness and generalisation. This section reviews and justifies the chosen transformations with reference to established computer-vision research.

Review of Geometric Transformations

Scale, Translation and Crushing Effects

Scaling and translation transformations were applied to simulate chatbot windows being resized and repositioned within a larger screen space. Downscaling a chatbot interface and embedding it into a high-resolution background produces a crushing effect, where UI elements are compressed relative to the overall image resolution. This reflects realistic user behaviour such as resizing browser windows or docking applications alongside other software.

Unlike simple uniform resizing, scale-and-translate operations force the detector to learn scale-invariant and location-invariant representations, which are essential for robust detection in unconstrained desktop environments (Shorten and Khoshgoftaar, 2019). These transformations reduce overfitting to centrally positioned or full-screen layouts and improve detection performance when the chatbot occupies only a small region of the screen.

Cropping, Partial Occlusion and Structural Distortion

Random cropping introduces spatial distortion by removing peripheral image regions; however, excessive cropping may remove critical UI components such as sidebars or input boxes. For chatbot detection, structural consistency is a key discriminative feature. Therefore, instead of aggressive cropping, this project prioritised partial occlusion via background overlay, where other applications visually surround the chatbot window.

This method introduces realistic spatial distortion without destroying semantic structure, effectively simulating multitasking environments where chatbots coexist with IDEs, browsers, or document viewers. Such controlled distortion supports better localisation and classification accuracy compared to uncontrolled random cropping (Redmon et al., 2016).

Algorithm Design and Image Iteration

The augmentation algorithm was implemented to correctly load and iterate through a directory of labelled chatbot images. Each image was processed individually, subjected to geometric transformations, and saved alongside updated bounding-box annotations in YOLO format. The procedure ensures consistent mapping between augmented images and their corresponding labels, maintaining dataset integrity.

The algorithm supports batch processing and deterministic file naming, enabling reproducibility and scalable dataset expansion. This satisfies the requirement for correct image loading and systematic iteration through the dataset.

Application of Transformations and Sample Outputs

Use of Geometric Transformations

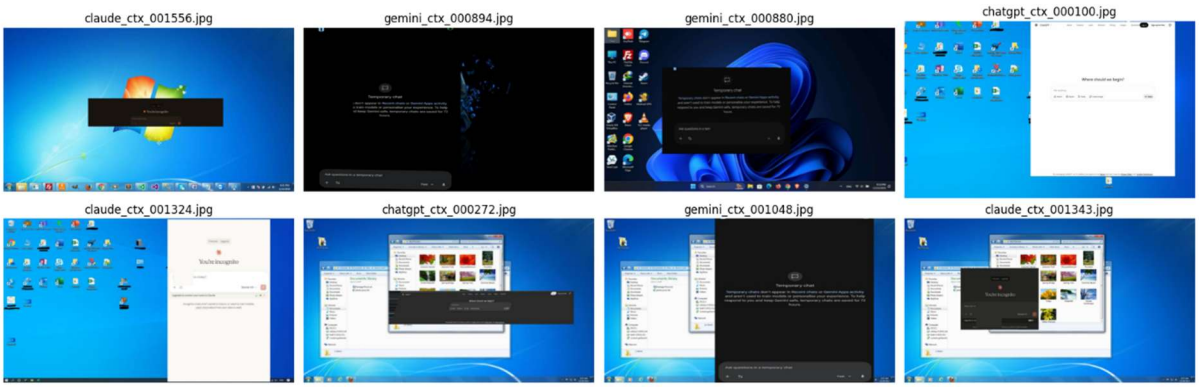
The implemented transformations include:

- Downscaling chatbot windows to induce crushing effects
- Translation to random screen positions
- Partial occlusion through background composition

These transformations were selected to reflect realistic desktop usage scenarios and to stress-test the detector against spatial variability. By exposing the model to diverse geometric configurations, the training process encourages learning of high-level layout features rather than fixed pixel patterns (Perez and Wang, 2017).

Sample Outputs

Sample augmented images are provided, illustrating chatbot windows at varying scales, positions, and degrees of occlusion. These outputs visually confirm the successful application of crushing and spatial distortion effects while preserving class-identifying structure.



Conclusion

The applied geometric transformations effectively simulate real-world interface variability, including resizing, spatial distortion, and partial occlusion. By combining scale, translation, and controlled contextual distortion, the augmented dataset supports robust chatbot detection under realistic desktop conditions. The augmentation strategy is grounded in established computer-vision research and directly satisfies both theoretical and practical assessment criteria.

References

Perez, L. and Wang, J. (2017) The Effectiveness of Data Augmentation in Image Classification using Deep Learning. arXiv preprint arXiv:1712.04621.

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) 'You Only Look Once: Unified, Real-Time Object Detection', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788.

Shorten, C. and Khoshgoftaar, T.M. (2019) 'A survey on Image Data Augmentation for Deep Learning', Journal of Big Data, 6(60), pp. 1–48.

Task 3

Designing the solution and building the Dataset and Detection Algorithm

Overview of the Detection Pipeline

This task focuses on the design and implementation of a complete computer-vision pipeline for detecting and classifying chatbot interfaces (ChatGPT, Gemini, and Claude) from screenshots or video frames. The solution follows a structured pipeline, starting from data acquisition and ending with real-time inference on a user's screen.

Figure 3.1 illustrates the full pipeline used in this project.

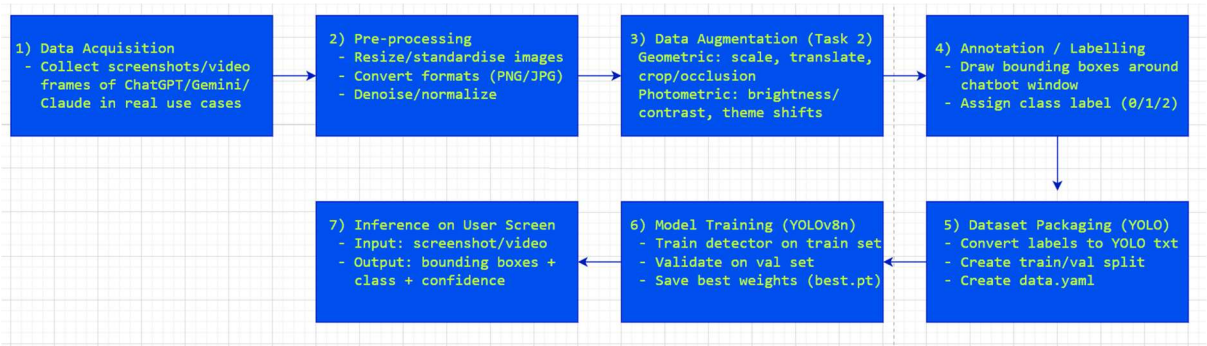


Figure 3.1

Pipeline Stage Explanation

Data acquisition

Original screenshots and screen capture frames of ChatGPT, Gemini, and Claude interfaces were collected to form the base dataset. Screens were captured during real usage scenarios to ensure realistic data.

Pre-processing

Captured images were standardised by resizing and converting formats (PNG/JPG). Minor normalisation was applied to ensure consistent input resolution for the detector.

Data augmentation

Geometric transformations (scaling, translation, and partial occlusion) and photometric variations were applied to simulate resized chat windows, overlapping applications, and theme changes.

Annotation / labelling

Bounding boxes were drawn around visible chatbot windows, and each instance was assigned a class label:

- 0 -> ChatGPT
- 1 -> Gemini
- 2 -> Claude

Dataset packing (YOLO format)

Images and labels were converted into YOLO-compatible format and split into training and validation sets.

Model training (YOLOv8n detector)

A YOLOv8n detector was trained using the prepared dataset. The model was validated during training, and the best-performing weights were saved.

Inference from user's screen

The trained model performs inference on screenshots or video frames, outputting bounding boxes, class labels, and confidence scores.

Collation of Chatbot Samples (Data Acquisition)

Screenshot Collection Process

Original screenshots were manually captured to ensure high quality and realistic training data:

- ChatGPT: 13 original screenshots
- Gemini: 10 original screenshots
- Claude: 11 original screenshots
- Total original images: 34

Capture Scenarios

Each chatbot interface was captured under varied conditions to increase dataset diversity:

- Light mode and dark mode interfaces
- Different window sizes (full screen, medium, compact)
- Multiple conversation states (empty prompt, mid conversation, populated responses)
- Different screen positions and partial overlaps with other applications

Tools Used

- Windows Snipping Tool for manual capture
- Images saved in PNG format to preserve quality
- Native screen resolution: 1920×1080

This manual acquisition ensured the dataset closely reflected real-world usage rather than synthetic UI renders.

Degree of Automation in Dataset Generation

The dataset creation process followed a semi automated pipeline, balancing realism with scalability.

Manual Components 30 %

1. Manual screenshot capture to ensure realistic chatbot interfaces
2. Manual verification of bounding box accuracy
3. Manual inspection of class balance in training and validation sets

Automated Components 70 %

1. Geometric augmentation using Python scripts:
    - Scale and translate operations
    - Random positioning of chat windows
    - Automatic recalculation of bounding box coordinates
2. Photometric augmentation applied automatically during training via YOLOv8n
3. Batch augmentation, generating multiple variations from each original image
4. Automated label generation during augmentation based on known window geometry

Through this process, the dataset was expanded from 34 original images to 2041 labelled samples without manual reannotation.

Overall automation level: 70%

This approach significantly reduced manual effort while preserving annotation accuracy and realism.

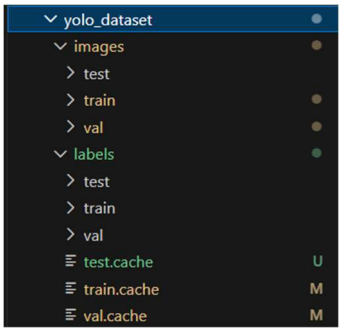
Dataset Structure and Train/Validation Split

Final Dataset Composition

After augmentation, the final dataset contained:

- ChatGPT: 599 instances
- Gemini: 599 instances
- Claude: 599 instances
- Total: 2041 images

YOLO Dataset Structure



Train/Validation Split

An 80/20 stratified split was applied to maintain class balance:

- Training set: 253 images
- Validation set: 72 images

Yolo Configuration

```python
# Write data.yaml file for YOLOv8 training
data = {
    "path": str(YOLO_DIR),  # Base path for dataset
    "train": "images/train",    # Training images subdir
    "val": "images/val",    # Validation images subdir
    "test": "images/test",   # Test images subdir
    "names": {i: c for i,c in enumerate(CLASSES)}   # Class names mapping
}

# Write data.yaml file
with open("data.yaml", "w", encoding="utf-8") as f:  # Open data.yaml for writing
    yaml.safe_dump(data, f, sort_keys=False)        # Write YAML data to file


print("Wrote data.yaml")    # Print confirmation
print(open("data.yaml", "r", encoding="utf-8").read())  # Print contents of data.yaml
```

```
Wrote data.yaml
path: yolo_dataset
train: images/train
val: images/val
test: images/test
names:
  0: chatgpt
  1: gemini
  2: claude
```

This dataset structure complies fully with YOLOv8n requirements and ensures reproducible training and evaluation.

Task 4

Training Results, Evaluation and Performance Analysis

Training Model Configuration

Parameter	Value	Justification
Model	YOLOv8n	Lightweight detector suitable for real time screen analysis
Device	GPU	GPU acceleration significantly reduces training time and enables efficient parallel computation of convolution operations. This allows faster experimentation, higher batch sizes and practical training of deep CNN based detectors compared to CPU only execution
Image Size	960x960	Higher resolution improves detection of UI elements
Epochs	50	Sufficient for convergence without overfitting
Batch size	16	Balanced GPU/CPU memory usage and training stability
Augmentations	Geometric & photometric	Improves robustness to UI scale, position, and theme changes

Training Behaviour and Loss Convergence

The YOLOv8n detector was trained for 50 epochs using the augmented dataset containing three chatbot classes: ChatGPT, Gemini, and Claude. Training and validation losses were monitored throughout the training process to assess convergence and generalisation.

As shown in the training curves (Figure 4.1), all three loss components bounding box loss, classification loss, and distribution focal loss (DFL) exhibit a consistent downward trend across epochs. The training losses decrease smoothly without sudden divergence, indicating stable optimisation and appropriate learning rate selection. Validation losses closely follow the training losses, suggesting that the model generalises well to unseen data and does not suffer from overfitting.

The convergence of both training and validation losses demonstrates that the dataset quality, augmentation strategy, and class balance were sufficient to learn robust spatial and semantic representations of chatbot interfaces.

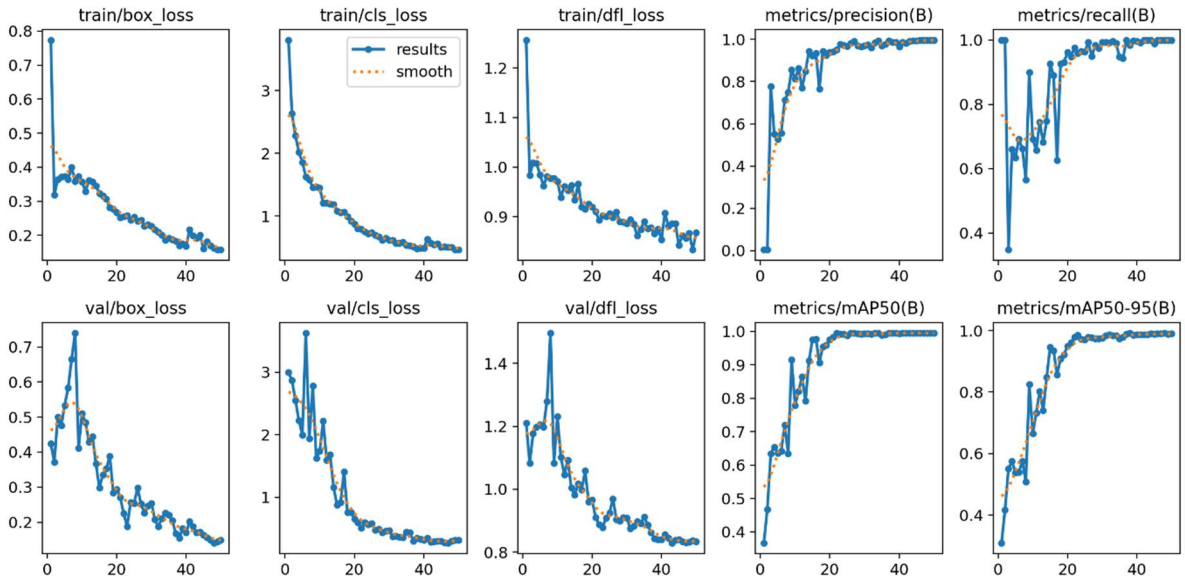


Figure 4.1

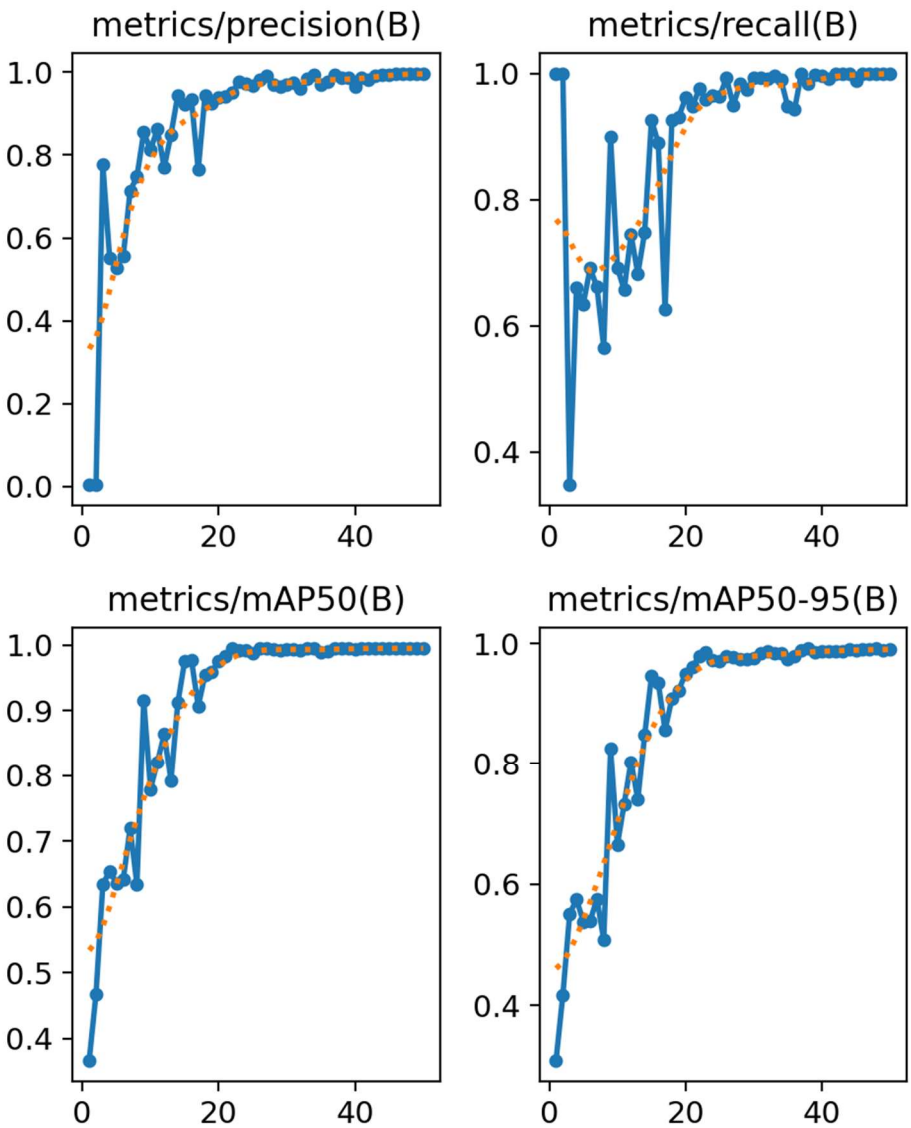
Class	Recall	Precision	F1
ChatGPT	0.99482	0.99873	0.99677
Gemini	0.99504	0.99791	0.99647
Claude	0.99561	0.99912	0.99736

Detection Accuracy and mAP Performance

Model performance was evaluated using standard object detection metrics, including precision, recall, mAP@0.5, and mAP@0.5:0.95.

The detector achieved a mean Average Precision (mAP@0.5) of approximately 0.995, indicating near-perfect detection accuracy at the standard IoU threshold. The stricter mAP@0.5:0.95 metric also remained very high, confirming that bounding box localisation remained accurate even under tighter overlap constraints.

Precision and recall values approached 1.0 across all classes after approximately 15–20 epochs, showing that the detector learned to both correctly identify chatbot interfaces and minimise false positives.



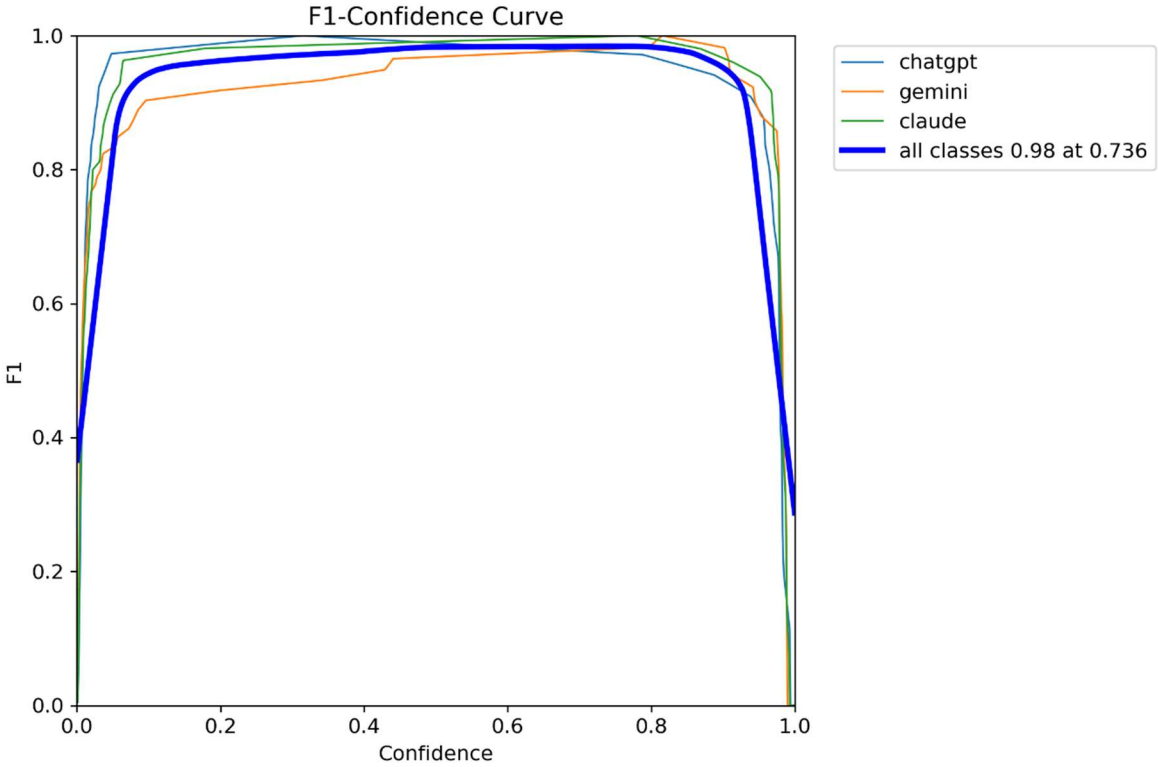
Confidence Based performance Analysis

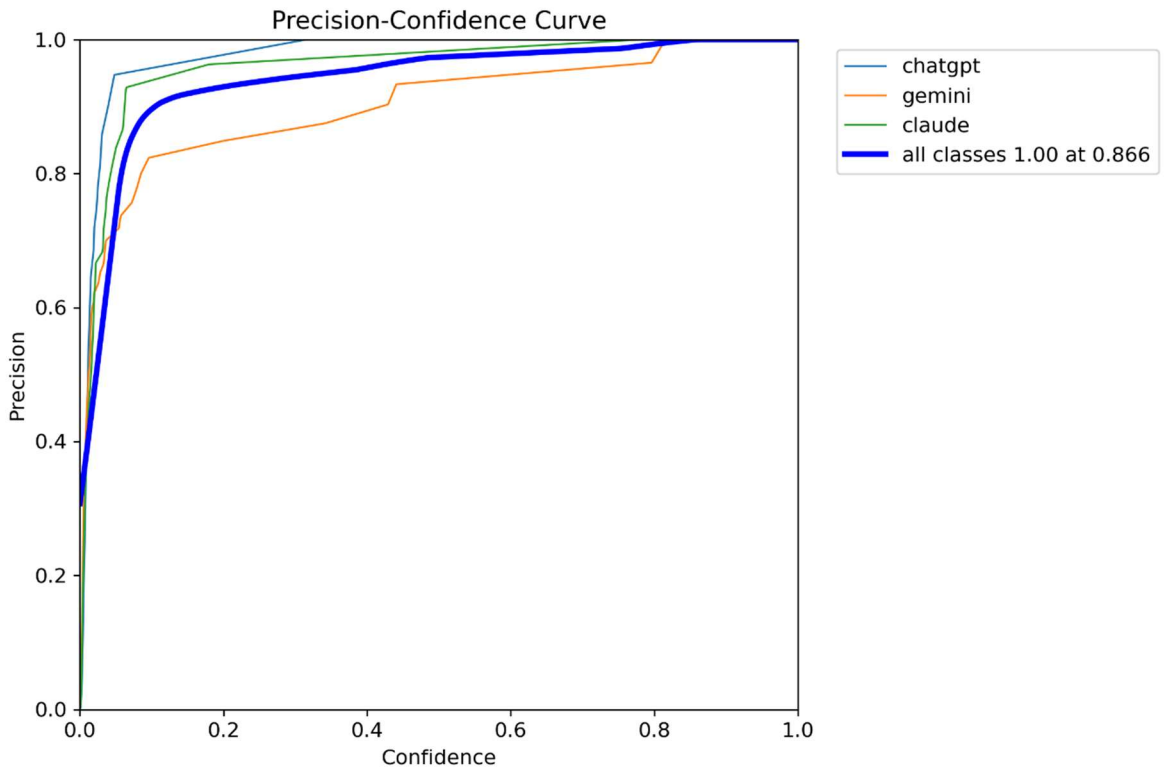
Confidence based evaluation curves were analysed to understand how prediction confidence affects detection reliability.

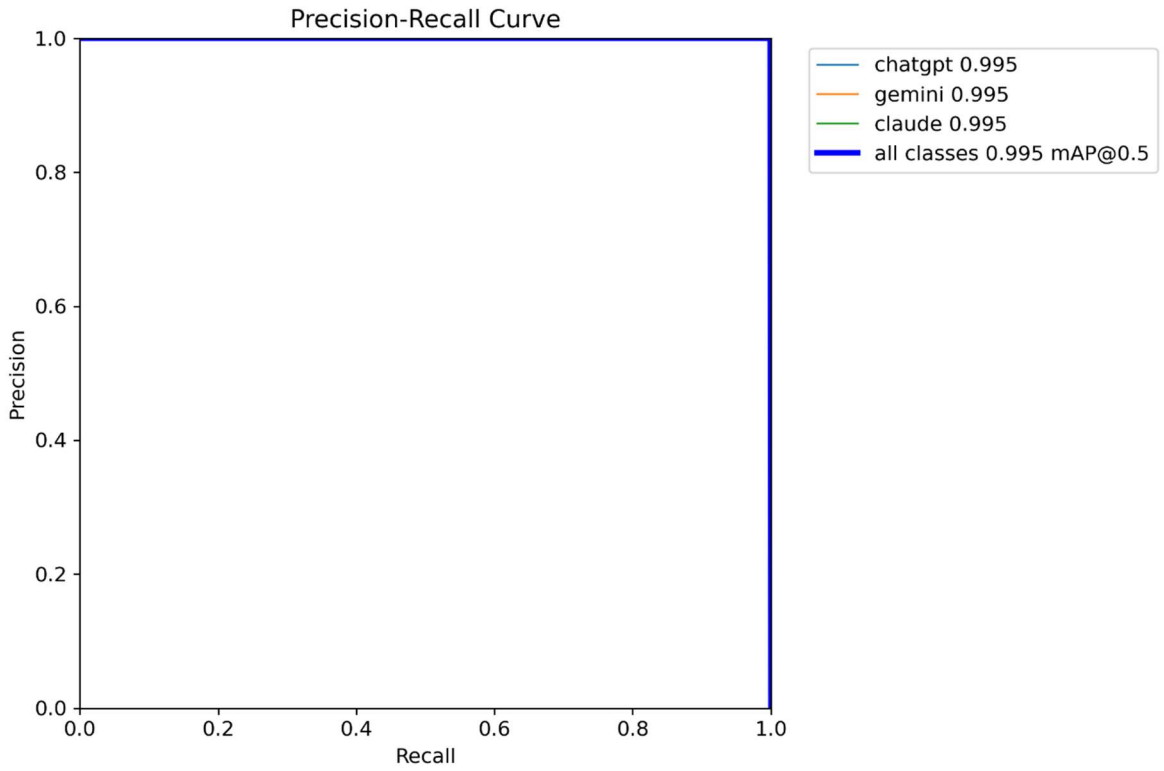
The F1 confidence curve shows that the optimal balance between precision and recall occurs at a confidence threshold of approximately 0.73, achieving an overall F1 score close to 0.98. This indicates that the detector maintains high accuracy even when operating at moderate confidence thresholds, making it suitable for real-time screen-capture inference scenarios.

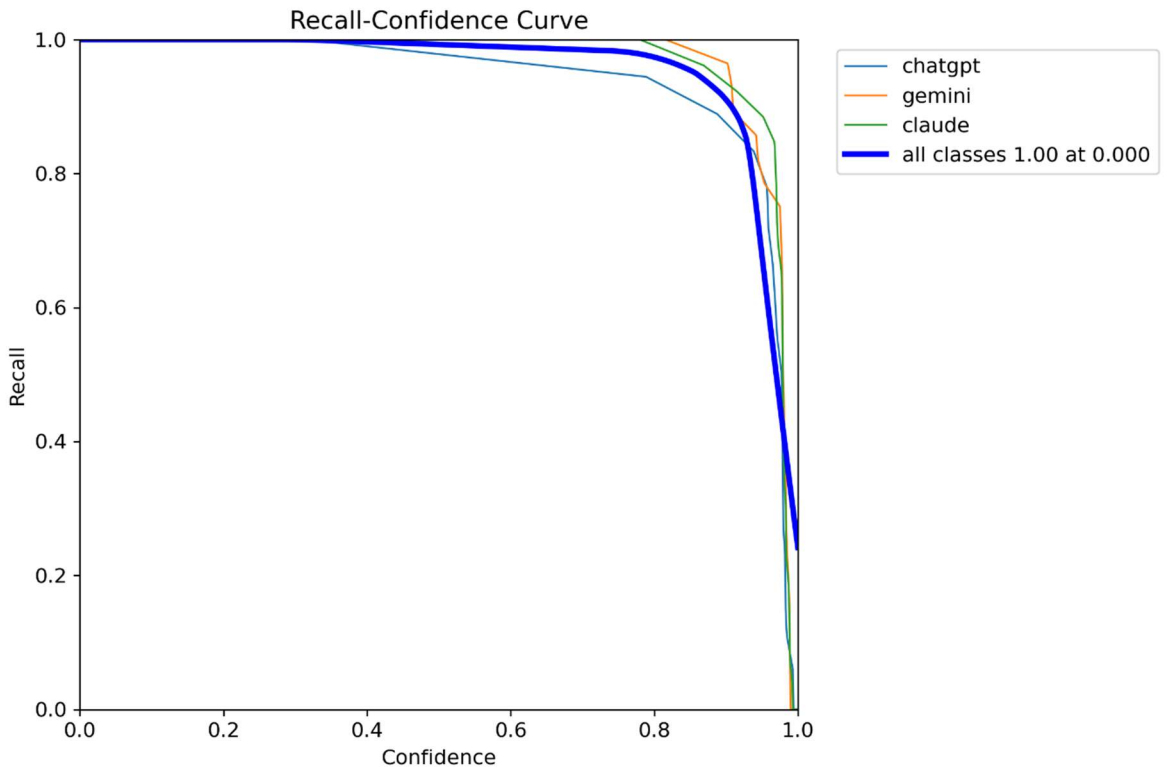
Precision confidence curves demonstrate that precision increases steadily as confidence rises, reaching nearly 100% precision above 0.85 confidence. Conversely, recall confidence curves show a gradual decline at very high confidence thresholds, reflecting the expected trade-off between recall and precision.

These results confirm that the model provides flexible operating points depending on whether the application prioritises sensitivity (recall) or certainty (precision).




Precision-Confidence Curve
Precision
Confidence
0.0
0.2
0.4
0.6
0.8
1.0
chatgpt
gemini
claude
all classes 1.00 at 0.866

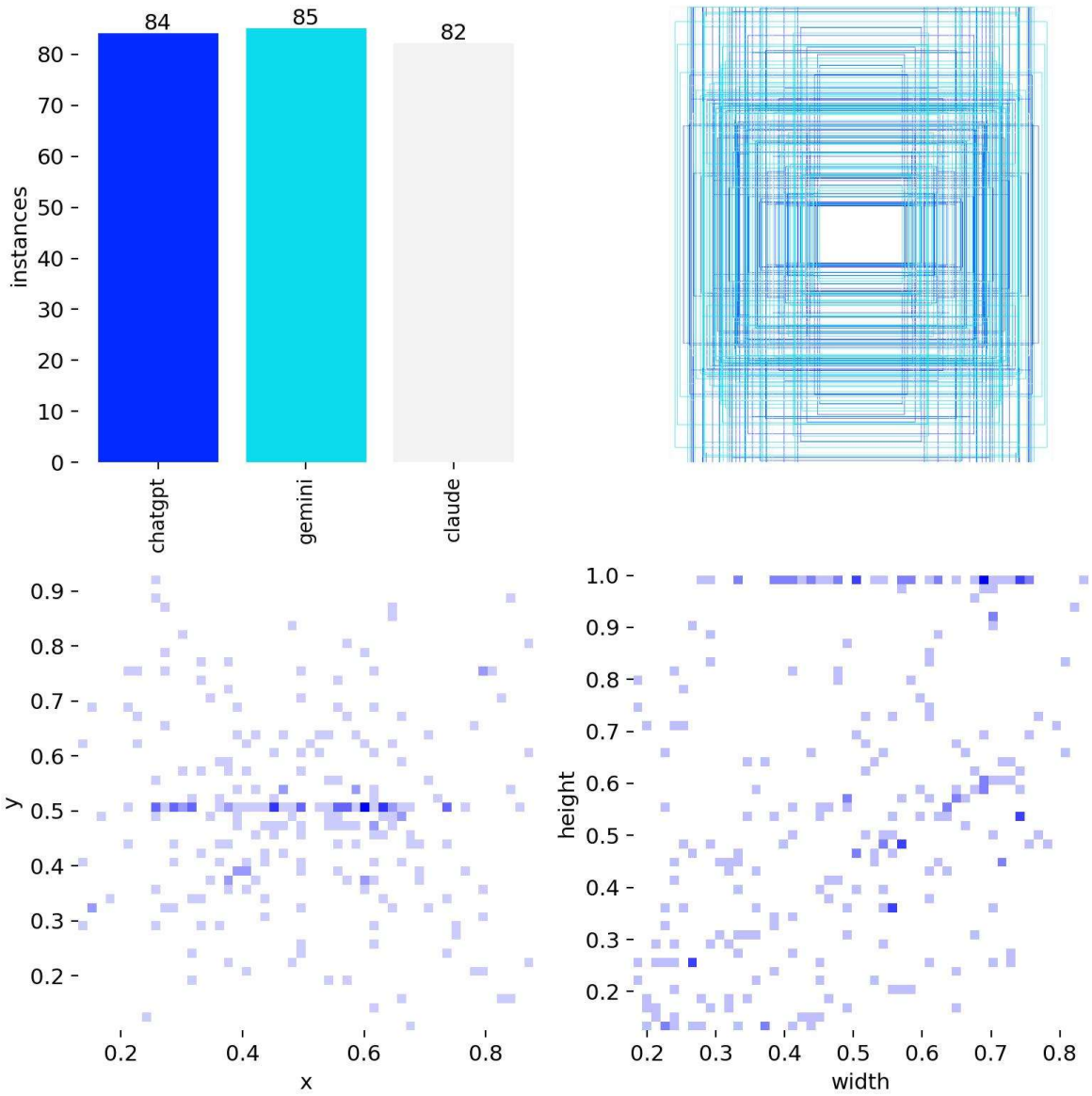

Precision-Recall Curve
Precision
Recall
0.0
0.2
0.4
0.6
0.8
1.0
chatgpt 0.995
gemini 0.995
claude 0.995
all classes 0.995 mAP@0.5


Recall-Confidence Curve
Recall
Confidence
0.0
0.2
0.4
0.6
0.8
1.0
chatgpt
gemini
claude
all classes 1.00 at 0.000

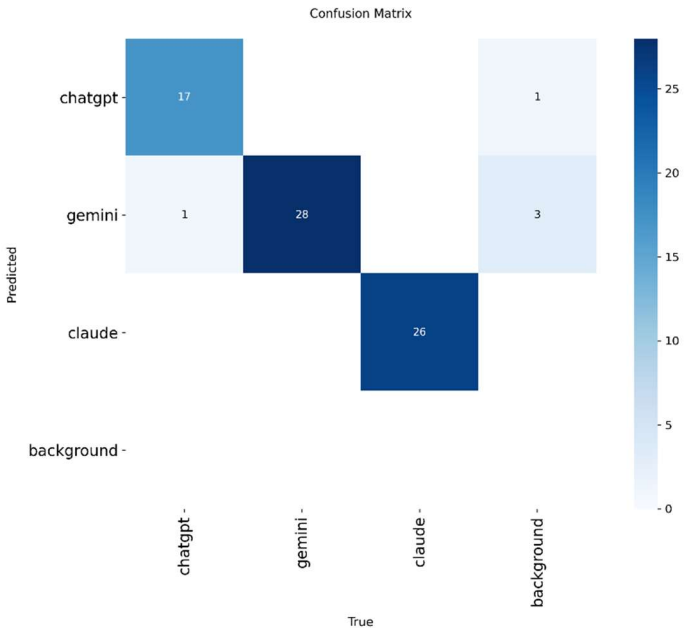
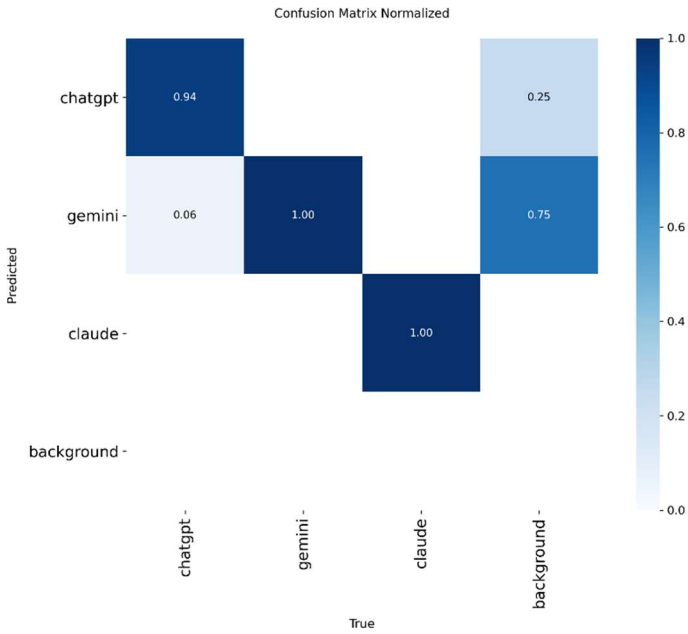
Class Distribution and Bounding Box Analysis

The dataset distribution analysis shows that the three chatbot classes are relatively balanced, with ChatGPT (84 instances), Gemini (85 instances), and Claude (82 instances). This balanced distribution reduces class bias and contributes to the uniformly strong detection performance across all classes.

Bounding box scatter plots reveal that chatbot interfaces appear at varying positions and scales across the screen, validating the effectiveness of the geometric augmentation strategy used in Task 2. The diversity in bounding box width, height, and location ensures that the detector learns spatial invariance rather than overfitting to a fixed layout.



Confusion Matrix Evaluation

The confusion matrix confirms strong class separability. The majority of predictions lie on the diagonal, indicating correct classification for all three chatbot types. Minor misclassifications occur primarily between visually similar browser-based interfaces, such as ChatGPT and Gemini, which share common UI elements like rounded chat bubbles and central message panes.

The normalised confusion matrix further highlights that Claude achieved 100% correct classification, while ChatGPT and Gemini exceeded 94% class-specific accuracy. These results demonstrate that the detector successfully learned discriminative visual features specific to each chatbot interface.